

## Лабораторная работа «Git: от локального репозитория до Pull Request»

Максимум: 10 баллов.

Каждая выполненная задача- 1 балл.

Задачи выполняются последовательно. Для локальной части в отчёт прикладываются скриншоты терминала. Для GitHub - ссылка на соответствующий объект + скриншот, если он требуется в задании. Скриншоты должны позволять определить репозиторий и результат выполнения задания.

| Б  | Задача                                   | Что необходимо сделать                                                                                                                                                                                                                                                                                                                   | Подтверждение                                                                       |
|----|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1  | Создать репозиторий                      | Создать локальный проект, инициализировать Git, настроить автора коммитов. Изучить появившуюся служебную структуру Git и определить, где хранятся HEAD, ветки и объекты.                                                                                                                                                                 | Скриншот локального репозитория и содержимого .git.                                 |
| 2  | Разобраться с состояниями файлов         | Создать несколько файлов и получить состояния untracked, staged и modified. Добиться ситуации, когда один файл одновременно имеет staged-версию и новые unstaged-изменения. Создать .gitignore. Отдельно проверить, что происходит, если игнорируемый файл уже начал отслеживаться Git.                                                  | Скриншоты состояний + краткое объяснение разницы между рабочей директорией и index. |
| 3  | Заглянуть внутрь Git                     | Создать несколько коммитов. Найти реальные Git-объекты своего репозитория: commit, tree и blob. Посмотреть их содержимое и SHA. Изменить содержимое одного файла и показать, что его hash изменился.                                                                                                                                     | Скриншот + SHA собственных commit, tree, blob.                                      |
| 4  | Поработать с историей и ветками          | Получить историю минимум из 5 осмысленных коммитов. Создать несколько веток и изменения в них. Хотя бы одну ветку объединить обычным merge, другую - через squash. Сравнить получившуюся историю.                                                                                                                                        | Скриншот графа истории Git и короткое объяснение разницы обычного merge и squash.   |
| 5  | Связать локальный Git с GitHub           | Создать GitHub-репозиторий и связать его с локальным. Выполнить первый push. После этого создать второй локальный clone этого же репозитория. Сделать изменение в первой копии и добиться ситуации, когда во второй копии удалённая ветка уже содержит новый commit, а локальная ещё нет. После этого синхронизировать её.               | Ссылка на репозиторий + скриншот состояния до синхронизации.                        |
| 6  | Подготовить GitHub workflow              | Создать собственный Pull Request Template в .github/. Настроить GitHub Actions. Защитить main: изменения должны попадать туда через PR, а merge должен зависеть от успешного CI. Проверить, что шаблон автоматически появляется при создании PR.                                                                                         | Ссылки на template и workflow + скриншот формы PR с шаблоном.                       |
| 7  | Создать PR и намеренно получить конфликт | Создать Issue, отдельную feature-ветку и Pull Request, связанный с Issue. После открытия PR самостоятельно создать конфликт между PR и main, чтобы GitHub запретил merge. Затем разобраться, как разрешить конфликт, и обновить существующий PR.                                                                                         | Ссылка на Issue и PR + скриншот PR в состоянии конфликта.                           |
| 8  | Пройти настоящее Code Review             | Провести review должен другой студент. Ревьюер оставляет минимум два содержательных inline-комментария, хотя бы один из которых требует изменения. Автор исправляет замечание новым commit и отвечает на комментарий. После проверки ревьюер делает Approve.                                                                             | Ссылка на PR, где видны review, исправление и approve.                              |
| 9  | Сломать CI секретом                      | Добавить в GitHub Actions поиск секретов через TruffleHog. Создать второй Issue, новую ветку и второй PR. Самостоятельно придумать изменение, которое TruffleHog сочтёт секретом. Добиться, чтобы CI завершился ошибкой и это было видно непосредственно в PR. Использовать только тестовые данные - реальные токены и пароли запрещены. | Ссылка на второй PR + ссылка на failed Actions run + скриншот красного CI в PR.     |
| 10 | Починить CI и завершить workflow         | Не создавая новый PR, исправить проблему из предыдущей миссии новым commit. Добиться успешного прохождения TruffleHog и остальных проверок. После зелёного CI завершить review/merge и убедиться, что связанный Issue закрылся. Показать итоговую историю репозитория.                                                                   | Ссылка на тот же PR, successful Actions run и merge + финальный граф Git.           |

### Требования к сдаче

- Один короткий отчёт PDF. Для миссий 1-5 - скриншоты локальной работы и необходимые короткие ответы. Для миссий 6-10 - ссылки на GitHub-артефакты и необходимые скриншоты.
- Не нужны большие теоретические описания. Основное доказательство выполнения работы - реальное состояние Git/GitHub, история коммитов, Pull Request, Code Review и запуски CI.
- Задача засчитывается только при наличии проверяемого результата. Конечное состояние репозитория без требуемой промежуточной истории выполнения задачи не засчитывается.